MIT Art Design and Technology University

MIT School of Computing, Pune

Database Management System
(23CSE2022)

Class - S.Y. (SEM-I)

Unit – II
Relational Algebra and Joins

AY 2026-2027 SEM-I



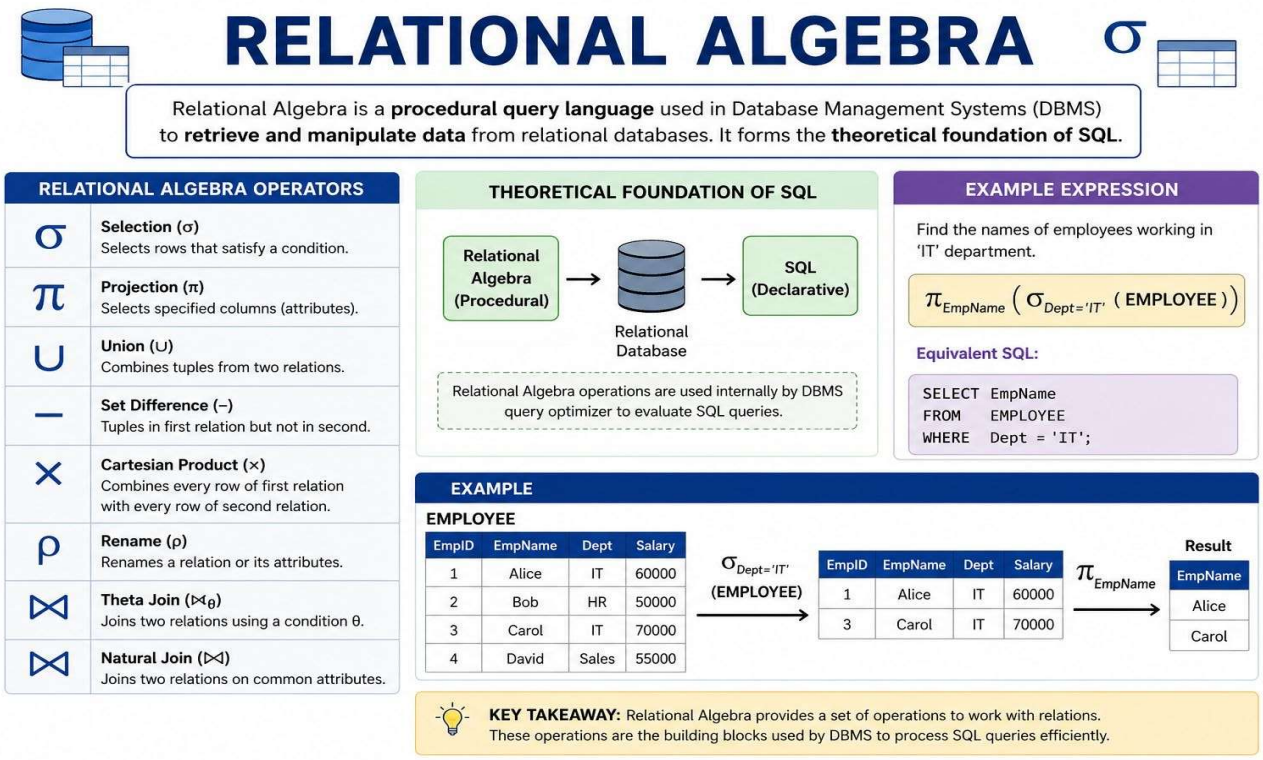
Unit - II Relational Algebra and Joins 09 hours

Relational algebra:
Introduction, Selection and projection, set operations, renaming, Joins, Division, syntax, semantics.
Operators, grouping and ungrouping, relational comparison.
Joins:
Inner Join, Outer Joins (Left, Right, Full), Theta Join, Natural Join, Cross Join

Introduction

Relational Algebra is a **procedural query language** used in **Database Management Systems (DBMS)** to retrieve and manipulate data from relational databases. It forms the theoretical foundation of SQL.

- It is a procedural query language.
- It operates on relations (tables).
- Each operation returns a new relation.
- It is the mathematical foundation of SQL.
- It is mainly used for query processing and optimization in DBMS.



Example of a Relation

account_number	*branch_name*	*balance*
A-101	Downtown	500
A-102	Perryridge	400
A-201	Brighton	900
A-215	Mianus	700
A-217	Brighton	750
A-222	Redwood	700
A-305	Round Hill	350

Basic Structure

- Formally, given sets D_1, D_2, …. D_n a **relation** *r* is a subset of
  D_1 x D_2 x … x D_n
  Thus, a relation is a set of *n*-tuples (a_1, a_2, …, a_n) where each $a_i \in D_i$
- Example: If
  - *customer_name* = {Jones, Smith, Curry, Lindsay, …} /* Set of all customer names */
  - *customer_street* = {Main, North, Park, …} /* set of all street names*/
  - *customer_city* = {Harrison, Rye, Pittsfield, …} /* set of all city names */

  Then *r* = { (Jones, Main, Harrison),
  (Smith, North, Rye),
  (Curry, North, Rye),
  (Lindsay, Park, Pittsfield) }

  is a relation over

  customer_name x customer_street x customer_city

Attribute Types

- Each attribute of a relation has a name
- The set of allowed values for each attribute is called the **domain** of the attribute
- Attribute values are (normally) required to be **atomic**; that is, indivisible
  - E.g. the value of an attribute can be an account number, but cannot be a set of account numbers
- Domain is said to be atomic if all its members are atomic
- The special value *null* is a member of every domain
- The null value causes complications in the definition of many operations
  - We shall ignore the effect of null values in our main presentation and consider their effect later

Relation Schema

- $A_1, A_2, \ldots, A_n$ are *attributes*

- $R = (A_1, A_2, \ldots, A_n)$ is a *relation schema*

  Example:

  Customer_schema = (*customer_name, customer_street, customer_city*)

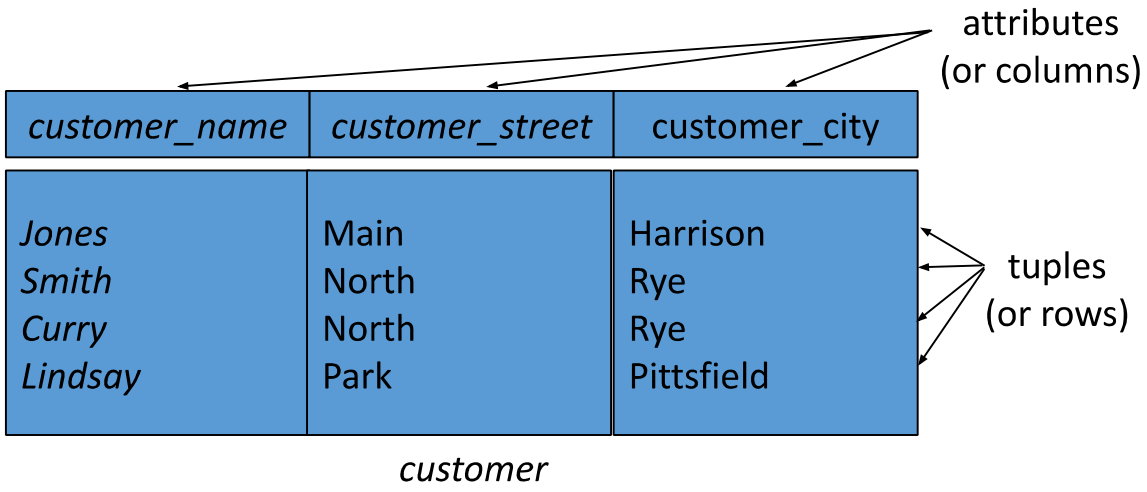
- *r*(*R*) denotes a *relation r* on the *relation schema R*

  Example:

  customer (Customer_schema)

Relation Instance

- The current values (*relation instance*) of a relation are specified by a table
- An element *t* of *r* is a *tuple*, represented by a *row* in a table



Relations are Unordered

Order of tuples is irrelevant (tuples may be stored in an arbitrary order)

Example: *account* relation with unordered tuples

account_number	*branch_name*	*balance*
A-101	Downtown	500
A-215	Mianus	700
A-102	Perryridge	400
A-305	Round Hill	350
A-201	Brighton	900
A-222	Redwood	700
A-217	Brighton	750

Database

- A database consists of multiple relations
- Information about an enterprise is broken up into parts, with each relation storing one part of the information
- account : stores information about accounts
  depositor : stores information about which customer owns which account
  customer : stores information about customers
- Storing all information as a single relation such as
  bank(account number, balance, customer name, ..)
  results in
  - repetition of information
    - e.g., if two customers own an account (What gets repeated?)
  - the need for null values
    - e.g., to represent a customer without an account
- Normalization theory (Chapter 7) deals with how to design relational schemas

The *customer* Relation

customer_name	*customer_street*	*customer_city*
Adams	Spring	Pittsfield
Brooks	Senator	Brooklyn
Curry	North	Rye
Glenn	Sand Hill	Woodside
Green	Walnut	Stamford
Hayes	Main	Harrison
Johnson	Alma	Palo Alto
Jones	Main	Harrison
Lindsay	Park	Pittsfield
Smith	North	Rye
Turner	Putnam	Stamford
Williams	Nassau	Princeton

The *depositor* Relation

customer_name	*account_number*
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Smith	A-215
Turner	A-305

Constraints:

- Constraints in nothing but conditions, means some condition has followed to retrieved data.
- Types of constraints:

1.key constraints-NOT NULL,unique value

2.Domain constraints- every attributes is bound to have a specific range of values.

3. Referential Integrity constraints- refer key (foreign key)

Keys in DBMS:

- Key is used to uniquely identify any record of data from the relation.
- Types of key :

1.Super key

2.Candidate key

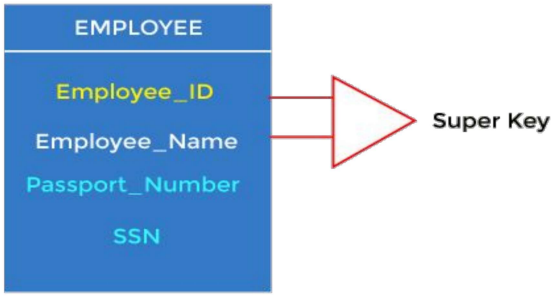
3.Primary key

4.Foreign key

5.Composite key

Super key:

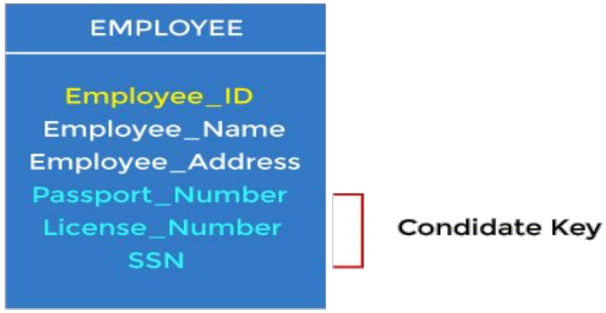
- A super key is combination of all possible attributes that can uniquely identify the row in the table.
- Super key is a superset of a candidate key.



For example: In the above EMPLOYEE table, for(EMPLOEE_ID, EMPLOYEE_NAME), the name of two employees can be the same, but their EMPLYEE_ID can't be the same. Hence, this combination can also be a key.

2.Candidate key

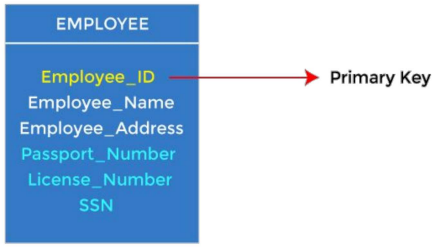
- A candidate key is an attribute or set of attributes that can uniquely identify a tuple.
- Except for the primary key, the remaining attributes are considered a candidate key. The candidate keys are as strong as the primary key.



- **For example:** In the EMPLOYEE table, id is best suited for the primary key. The rest of the attributes, like SSN, Passport_Number, License_Number, etc., are considered a candidate key.

3.Primary key

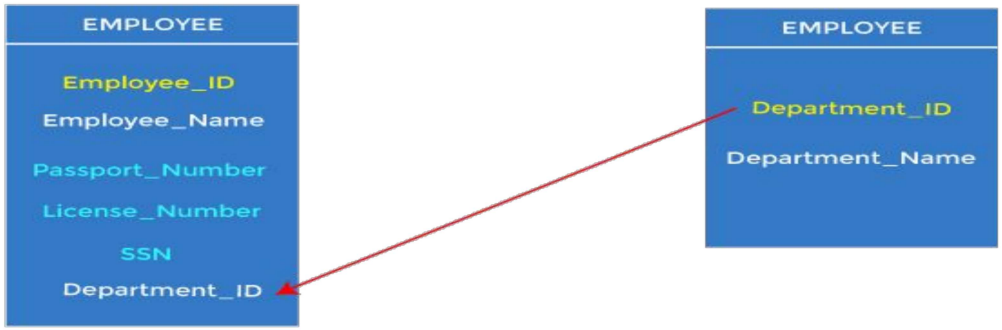
- It is the first key used to identify one and only one instance of an entity uniquely.



- In the EMPLOYEE table, ID can be the primary key since it is unique for each employee. In the EMPLOYEE table, we can even select License_Number and Passport_Number as primary keys since they are also unique.
- For each entity, the primary key selection is based on requirements and developers.

4.Foreign key

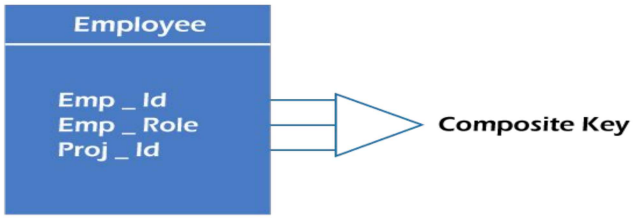
- Foreign keys are the column of the table used to point to the primary key of another table.



- Every employee works in a specific department in a company, and employee and department are two different entities. So we can't store the department's information in the employee table. That's why we link these two tables through the primary key of one table.
- We add the primary key of the DEPARTMENT table, Department_Id, as a new attribute in the EMPLOYEE table.
- In the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.

5.Composite key

- Whenever a primary key consists of more than one attribute, it is known as a composite key. This key is also known as Concatenated Key.



- **For example,** in employee relations, we assume that an employee may be assigned multiple roles, and an employee may work on multiple projects simultaneously. So the primary key will be composed of all three attributes, namely Emp_ID, Emp_role, and Proj_ID in combination. So these attributes act as a composite key since the primary key comprises more than one attribute.

Keys

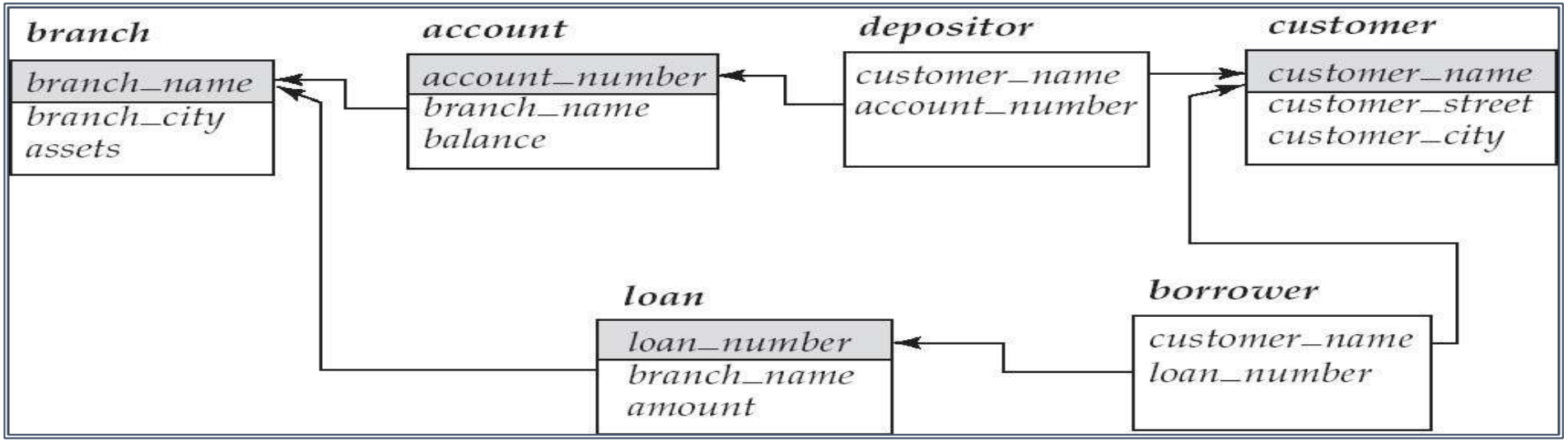
- Let K ⊆ R
- *K* is a **superkey** of *R* if values for *K* are sufficient to identify a unique tuple of each possible relation *r(R)*
  - by "possible *r* " we mean a relation *r* that could exist in the enterprise we are modeling.
  - Example: {*customer_name, customer_street*} and

    {*customer_name*}

    are both superkeys of *Customer*, if no two customers can possibly have the same name
    - In real life, an attribute such as *customer_id* would be used instead of *customer_name* to uniquely identify customers, but we omit it to keep our examples small, and instead assume customer names are unique.

Foreign Keys

- A relation schema may have an attribute that corresponds to the primary key of another relation. The attribute is called a **foreign key**.
  - E.g. *customer_name* and *account_number* attributes of *depositor* are foreign keys to *customer* and *account* respectively.
  - Only values occurring in the primary key attribute of the **referenced relation** may occur in the foreign key attribute of the **referencing relation.**
- **Schema diagram**



Query Languages

- Language in which user requests information from the database.
- Categories of languages
  - Procedural
  - Non-procedural, or declarative
- "Pure" languages:
  - Relational algebra
  - Tuple relational calculus
  - Domain relational calculus
- Pure languages form underlying basis of query languages that people use.

Relational Algebra

- Relational algebra is a procedural query language.
- It uses operators to perform queries.
- They accept relations as their input and yield relations as their output.
- Six basic operators
  - select: σ
  - project: ∏
  - union: ∪
  - set difference: –
  - Cartesian product: x
  - rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result.

Selection (σ)

Selection is used to select required tuples of the relations

- Syntax

  σcondition(Relation)

Example: **Student Table**

Roll_No	Name	Marks	City
101	Amit	85	Pune
102	Priya	90	Mumbai
103	Rahul	70	Pune

Query

Display students whose marks are greater than 80.

Relational Algebra:

σMarks>80(Student)

Roll_No	Name	Marks	City
101	Amit	85	Pune
102	Priya	90	Mumbai

2. Projection (π)

- **Definition**
  Projection (π) is used to **select specific columns (attributes)** from a relation.
- Syntax:

  πAttribute1,Attribute2(Relation)
- **Example:Student Table**

Roll_No	Name	Marks	City
101	Amit	85	Pune
102	Priya	90	Mumbai
103	Rahul	70	Pune

Query
Display only **Name** and **Marks.**

Relational Algebra

πName,Marks(Student)

Name	Marks
Amit	85
Priya	90
Rahul	70

3. Union (∪)

- **Definition**

The **Union (∪)** operation combines all tuples from **two relations**.

Conditions for Union:

- Both relations must have the **same number of attributes**.
- Corresponding attributes must have the **same data type**.
- **Simple Definition:**
  Union combines records from two tables and removes duplicate rows.
- Syntax

  sRelation1 ∪ Relation2

- Example: Student A

Roll_No	Name
101	Amit
102	Priya

- Student B

Roll_No	Name
102	Priya
103	Rahul

Relational Algebra

Student_A ∪ Student_B

Result: Duplicate record **Priya** appears only once.

Roll_No	Name
101	Amit
102	Priya
103	Rahul

4. Set Difference (−)

- **Definition**
  Set Difference (−) returns tuples that are present in the **first relation but not in the second**.
- Condition:
  Both relations must have the same attributes
  Attribute data types must match.
- **Simple Definition:**
  Set Difference finds records that exist in one table but not in another.
- **Syntax**

  Relation1−Relation2

Example

Student_A

Roll_No	Name
101	Amit
102	Priya
103	Rahul

Student B:

Roll_No	Name
102	Priya

Relational Algebra

Student_A − Student_B

Result:

Roll_No	Name
101	Amit
103	Rahul

5. Rename (ρ)

- **Definition**

The **Rename (ρ)** operation changes the **name of a relation or its attributes**.

Simple Definition:
Rename is used to give a new name to a table or column.

- **Syntax**

Rename a Relation(Relation is nothing but the table)

ρNewRelationName(Relation)

Rename Attributes

ρNewRelation(Attribute1,Attribute2,…)(Relation)

Example 1: Rename Relation

Student Table:

Roll_No	Name	Marks

Rename **Student** to **Stud**.

- **Relational Algebra**

  ρStud(Student)

Now the relation name becomes **Stud**.

Example 2: Rename Attributes

Rename **Roll_No** to **Student_ID** and **Name** to **Student_Name**.

Relational Algebra

ρStudent(Student_ID,Student_Name,Marks)(Student)

Result:

Student_ID	Student_Name	Marks
101	Amit	85
102	Priya	90

Cartesian-Product Operation – Example

Relations *r, s*:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

r x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Cartesian-Product Operation

- Cross product between two relations let say A and B, so cross product between A X B will results all the attributes of A followed by each attribute of B.
- Notation r x s
- Defined as:

  $r \times s = \{t\, q \mid t \in r \textbf{ and } q \in s\}$

- Assume that attributes of r(R) and s(S) are disjoint. (That is, $R \cap S = \varnothing$).
- If attributes of *r(R)* and *s(S)* are not disjoint, then renaming must be used.

Composition of Operations

- Can build expressions using multiple operations
- Example: $\sigma_{A=C}(r \times s)$
- $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

- $\sigma_{A=C}(r \times s)$

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Banking Example

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)

Example Queries

- Find all loans of over $1200

$$\sigma_{amount > 1200}(loan)$$

- Find the loan number for each loan of an amount greater than $1200
-

$$\prod_{loan_number}(\sigma_{amount > 1200}(loan))$$

- Find the names of all customers who have a loan, an account, or both, from the bank

$$\prod_{customer_name}(borrower) \cup \prod_{customer_name}(depositor)$$

Example Queries

- Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{customer_name}(\sigma_{branch_name=\text{“Perryridge”}}(\sigma_{borrower.loan_number = loan.loan_number}(borrower \times loan)))$$

Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$$\Pi_{customer_name}(\sigma_{branch_name = \text{“Perryridge”}}(\sigma_{borrower.loan_number = loan.loan_number}(\text{borrower} \times \text{loan}))) - \Pi_{customer_name}(\text{depositor})$$

Example Queries

- Find the names of all customers who have a loan at the Perryridge branch.

Query 1

$$\Pi_{customer_name} (\sigma_{branch_name = \text{“Perryridge”}} (\sigma_{borrower.loan_number = loan.loan_number} (borrower \times loan)))$$

Query 2

$$\Pi_{customer_name} (\sigma_{loan.loan_number = borrower.loan_number} ((\sigma_{branch_name = \text{“Perryridge”}} (loan)) \times borrower))$$

Example Queries

- Find the largest account balance
  - Strategy:
    - Find those balances that are *not* the largest
      - Rename *account* relation as *d* so that we can compare each account balance with all others
    - Use set difference to find those account balances that were *not* found in the earlier step.
  - The query is:

$$\prod_{balance}(account) - \prod_{account.balance} (\sigma_{account.balance < d.balance} (account \times \rho_{d} (account)))$$

Formal Definition

- A basic expression in the relational algebra consists of either one of the following:
  - A relation in the database
  - A constant relation
- Let E_1 and E_2 be relational-algebra expressions; the following are all relational-algebra expressions:
  - $E_1 \cup E_2$
  - $E_1 - E_2$
  - $E_1 \times E_2$
  - $\sigma_p (E_1)$, P is a predicate on attributes in E_1
  - $\prod_s (E_1)$, S is a list consisting of some of the attributes in E_1
  - $\rho_x (E_1)$, x is the new name for the result of E_1

Division Operation

- Notation: $r \div s$
- Suited to queries that include the phrase “for all”.
- Let *r* and *s* be relations on schemas *R* and *S* respectively where
  - $R = (A_1, \ldots, A_m, B_1, \ldots, B_n)$
  - $S = (B_1, \ldots, B_n)$

The result of r ÷ s is a relation on schema

$R - S = (A_1, \ldots, A_m)$

$$r \div s = \{ t \mid t \in \prod_{R-S}(r) \wedge \forall u \in s (tu \in r) \}$$

Where *tu* means the concatenation of tuples *t* and *u* to produce a single tuple

Division Operation – Example

Relations *r, s*:

A	B
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
∈	6
∈	1
β	2

r

B
1
2

s

r ÷ *s*:

A
α
β

Another Division Example

Relations *r, s*:

A	*B*	*C*	*D*	*E*
α	a	α	a	1
α	a	γ	a	1
α	a	γ	b	1
β	a	γ	a	1
β	a	γ	b	3
γ	a	γ	a	1
γ	a	γ	b	1
γ	a	β	b	1

r

D	*E*
a	1
b	1

s

r ÷ *s*:

A	*B*	*C*
α	a	γ
γ	a	γ

Division Operation (Cont.)

- Property
  - Let $q = r \div s$
  - Then q is the largest relation satisfying $q \times s \subseteq r$
- Definition in terms of the basic algebra operation
  Let $r(R)$ and $s(S)$ be relations, and let $S \subseteq R$

$$r \div s = \prod_{R-S}(r) - \prod_{R-S}((\prod_{R-S}(r) \times s) - \prod_{R-S,S}(r))$$

To see why

- $\prod_{R-S,S}(r)$ simply reorders attributes of r
- $\prod_{R-S}(\prod_{R-S}(r) \times s) - \prod_{R-S,S}(r))$ gives those tuples t in $\prod_{R-S}(r)$ such that for some tuple $u \in s$, $tu \notin r$.

Assignment Operation

- The assignment operation (←) provides a convenient way to express complex queries.
  - Write query as a sequential program consisting of
    - a series of assignments
    - followed by an expression whose value is displayed as a result of the query.
  - Assignment must always be made to a temporary relation variable.
- Example: Write $r \div s$ as

$$temp1 \leftarrow \prod_{R-S} (r)$$

$$temp2 \leftarrow \prod_{R-S} ((temp1 \times s) - \prod_{R-S,S} (r))$$

$$result = temp1 - temp2$$

  - The result to the right of the ← is assigned to the relation variable on the left of the ←.
  - May use variable in subsequent expressions.

Bank Example Queries

Find the names of all customers who have a loan and an account at bank.

$$\prod_{customer_name}(borrower) \cap \prod_{customer_name}(depositor)$$

- Find the name of all customers who have a loan at the bank and the loan amount

$$\prod_{customer_name,\ loan_number,\ amount}(borrower \bowtie loan)$$

Bank Example Queries

- Find all customers who have an account from at least the “Downtown” and the Uptown” branches.

Query 1

$$\Pi_{customer_name}(\sigma_{branch_name = \text{“Downtown”}}(depositor \bowtie account)) \cap \Pi_{customer_name}(\sigma_{branch_name = \text{“Uptown”}}(depositor \bowtie account))$$

Query 2

$$\Pi_{customer_name,\ branch_name}(depositor \bowtie account) \div \rho_{temp(branch_name)}(\{(\text{“Downtown”}), (\text{“Uptown”})\})$$

Note that Query 2 uses a constant relation.

Bank Example Queries

- Find all customers who have an account at all branches located in Brooklyn city.

$$\prod_{customer_name,\ branch_name} (depositor \bowtie account)$$
$$\div \prod_{branch_name} (\sigma_{branch_city = \text{“Brooklyn”}} (branch))$$

Aggregate Functions and Operations



- **Aggregation function** takes a collection of values and returns a single value as a result.

  avg: average value
  min: minimum value
  max: maximum value
  sum: sum of values
  count: number of values

- **Aggregate operation** in relational algebra

$$G_1, G_2, \ldots, G_n \, \mathcal{G}_{F_1(A_1), F_2(A_2, \ldots, F_n(A_n)}(E)$$

E is any relational-algebra expression

- $G_1, G_2 \ldots, G_n$ is a list of attributes on which to group (can be empty)
- Each F_i is an aggregate function
- Each A_i is an attribute name

Aggregate Function:

- An aggregate function in SQL **returns one value after calculating multiple values of a column.**
- We often use aggregate functions with the GROUP BY and HAVING clauses of the SELECT statement.
- **Count()**
- **Max()**
- **Min()**
- **Avg()**
- **Sum()**

Aggregate Function:

- An aggregate function in SQL **returns one value after calculating multiple values of a column.**
- We often use aggregate functions with the GROUP BY and HAVING clauses of the SELECT statement.
- **Count()**
- **Max()**
- **Min()**
- **Avg()**
- **Sum()**

GROUP BY – SYNTEX

```
SELECT column_name
FROM table_name
Where condition
GROUP BY column_name
ORDER BY column_name;
```

HAVING CLAUSE

- Having clause utilized in sql as a conditional clause with GROUP BY clause.
- Having clause restrict the data on the group records rather than individual records.
- A query may have both the clause (where & having clause)
- Where clause applied first and then having clause.

Having clause:

- Syntex:
- SELECT *column_name(s)*
  FROM *table_name*
  WHERE *condition*
  GROUP BY *column_name(s)*
  HAVING *condition*
  ORDER BY *column_name(s);*

Aggregate Operation – Example

- Relation *r*:

A	*B*	*C*
α	α	7
α	β	7
β	β	3
β	β	10

- $g_{\text{sum(c)}}(r)$

sum(*c*)
27

Aggregate Operation – Example



- Relation *account* grouped by *branch-name*:

branch_name	*account_number*	*balance*
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$$branch_name\ g_{\mathbf{sum}(balance)}\ (account)$$

branch_name	**sum**(*balance*)
Perryridge	1300
Brighton	1500
Redwood	700

Aggregate Functions (Cont.)

- Result of aggregation does not have a name
  - Can use rename operation to give it a name
  - For convenience, we permit renaming as part of aggregate operation

$$branch_name\ \mathcal{G}\ \textbf{sum}(balance)\ \textbf{as}\ sum_balance\ (account)$$

Joins

- The join operations is one of the most useful and commonly used operations to extract information from two or more relations.
- The join operations denoted by ⋈ and it is used to join two relations to form new relations on the basis of common attributes present in two relations.
- Types of Join

1. Inner Join
2. Outer Join ------ 2.1 left join
   2.2 Right Join
   2.3 Full Join
3. Theta Join
4. Natural Join
5. Cross Join

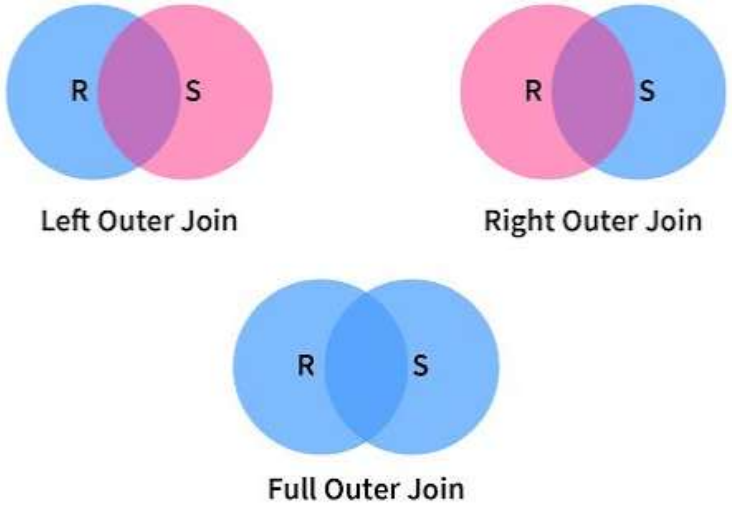
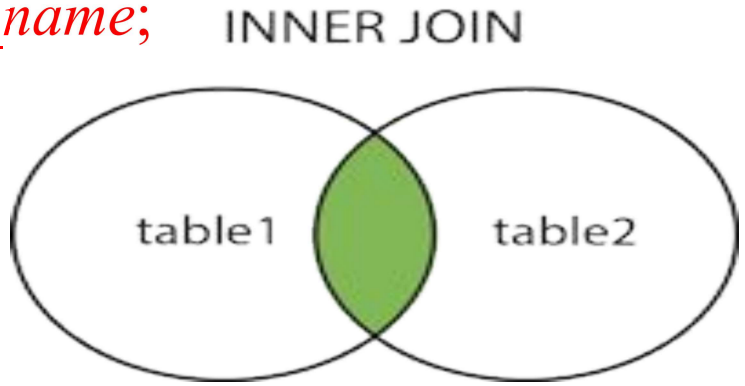


Fig. Outer join type

Inner Join

- The Inner join keyword select records that have matching values in both tables.
- **Syntax:**
- SELECT *column_name(s)*
  FROM *table1*
  INNER JOIN *table2*
  ON *table1.column_name = table2.column_name*;



Examples: SQL statement selects all orders with customer information

"Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10308	2	7	1996-09-18	3
10309	37	3	1996-09-19	1
10310	77	8	1996-09-20	2

"Customers" table

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

Inner Join :

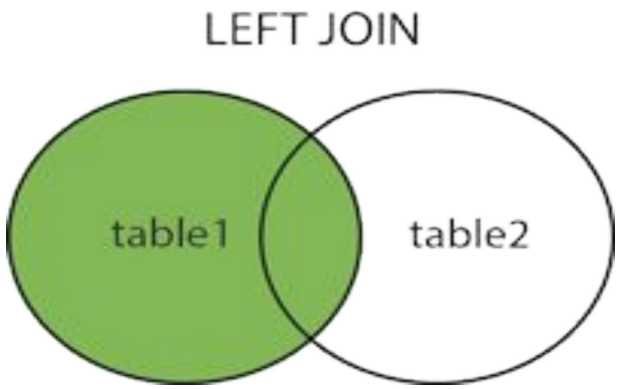
```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers
 ON Orders.CustomerID = Customers.CustomerID;
```

Left Join

- The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2).

Syntax:

- SELECT *column_name(s)*
  FROM *table1*
  LEFT JOIN *table2*
  ON *table1.column_name = table2.column_name*;



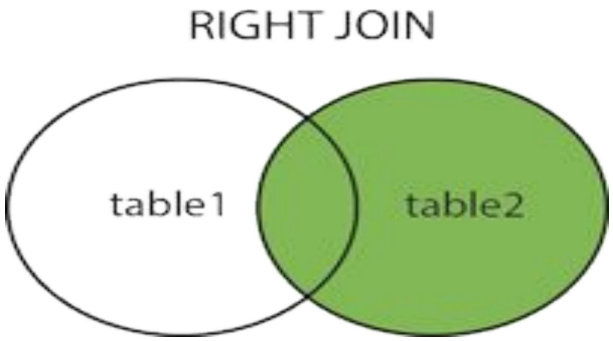
Left Join:

- SELECT Customers.CustomerName, Orders.OrderID
  FROM Customers
  LEFT JOIN Orders

ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;

Right Join

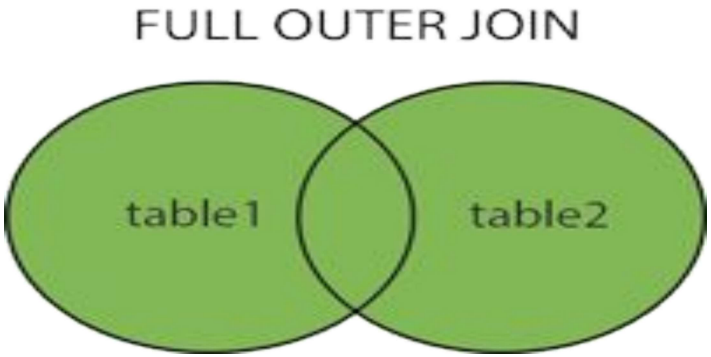
- The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1).
- Syntax:
- SELECT *column_name(s)*
  FROM *table1*
  RIGHT JOIN *table2*
  ON *table1.column_name = table2.column_name*;



- Right Join:
- SELECT Orders.OrderID, Employees.LastName, Employees.FirstName
  FROM Orders
  RIGHT JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
  ORDER BY Orders.OrderID;

Full Outer Join:

- The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records
- Syntax:
- SELECT *column_name(s)*
  FROM *table1*
  FULL OUTER JOIN *table2*
  ON *table1.column_name = table2.column_name*
  WHERE *condition;*



FULL OUTER JOIN can potentially return very large result-sets

- Full Outer Join:
- SELECT Customers.CustomerName,
  Orders.OrderID
  FROM Customers
  FULL OUTER JOIN Orders ON Customers.Customer
  ID=Orders.CustomerID
  ORDER BY Customers.CustomerName;

Self Join:

- A self join is a regular join, but the table is joined with itself. Syntax:
- SELECT *column_name(s)*
  FROM *table1 T1, table1 T2*
  WHERE *condition*;
- Examples:Customers" table:SQL statement matches customers that are from the same city

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico

- SELECT A.CustomerName AS CustomerName1, B.CustomerName AS CustomerName2, A.City
  FROM Customers A, Customers B
  WHERE A.CustomerID <> B.CustomerID
  AND A.City = B.City
  ORDER BY A.City;

Cross Join:

- Cross join used to combine each row of one table with each row of another table, and return the cross product of the sets of rows from the tables that are joined.

Syntax:

```
Select table1. column name,
table2.column name
 from table1
Cross Join table 2;
```

Natural Join:

- Two column name same.
- Common attributes should be equal.
- Join=(cross product + condition)

- Syntax:
- Select Coulmn Name From table 1 Natural Join Table2

Benefits and Limitations

- **Benefits:**
- Ensures data consistency by including only related records
- Simplifies complex queries by combining related data
- **Limitations:**
- Excludes non-matching records, which might be necessary in some analyses
- Can lead to loss of information if there are no matches

Aspect	INNER Join	Natural Join
Definition	Combines rows from two or more tables based on a specified condition.	Automatically combines rows from two tables based on columns with the same name and compatible data types.
Syntax	`` `sql SELECT column_name(s) FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name;` ``	`` `sql SELECT * FROM table1 NATURAL JOIN table2;` ``
Join Condition	Explicitly specified in the query using the `ON` clause.	Implicitly determined by matching column names and data types in both tables.
Flexibility	Highly flexible; allows joining on any condition, including multiple conditions and different operators (e.g., =, >, <).	Less flexible; only joins on columns with the same name and compatible data types.
Column Names	You need to specify the join condition, which can be any columns from the two tables.	Only works when tables have one or more columns with the same name and data types.
Result Columns	You can select specific columns or all columns from the joined tables.	Automatically includes all columns from both tables, without the need to specify column names.

Join Type	Definition	Syntax (Relational Algebra)	Key Features	Use Cases
Theta Join	Links tables based on a condition involving comparison operators.	`` `R ⋈θ S` ``	Flexible, condition-based join using operators like =, !=, >, <	Custom joins, complex conditions
Natural Join	Automatically joins tables based on columns with the same name and compatible data types.	`` `R ⋈ S` ``	Simplified syntax, no need to explicitly specify join condition	Simplifying queries where column names match
Cross Join	Produces the Cartesian product of two tables, combining all rows from the first table with all rows from the second table.	`` `R × S` ``	Cartesian product, often generates large datasets	Combinations, testing, data generation

JOIN TYPE	DEFINITION	JOIN CONDITION	RETURNS	UNMATCHED ROWS	EXAMPLE SYNTAX
INNER JOIN	Returns only the rows that have matching values in both tables.	ON Table1.column = Table2.column	Only matching rows	Excluded	SELECT * FROM Employee E INNER JOIN Department D ON E.DeptID = D.DeptID;
LEFT OUTER JOIN (LEFT JOIN)	Returns all rows from the left table and matching rows from the right table.	ON Table1.column = Table2.column	All left table rows + matching right table rows	Left table rows are retained; right-side values become NULL if no match	SELECT * FROM Employee E LEFT JOIN Department D ON E.DeptID = D.DeptID;
RIGHT OUTER JOIN (RIGHT JOIN)	Returns all rows from the right table and matching rows from the left table.	ON Table1.column = Table2.column	All right table rows + matching left table rows	Right table rows are retained; left-side values become NULL if no match	SELECT * FROM Employee E RIGHT JOIN Department D ON E.DeptID = D.DeptID;
FULL OUTER JOIN (FULL JOIN)	Returns all rows from both tables.	ON Table1.column = Table2.column	Matching rows + all unmatched rows from both tables	Unmatched rows from both tables appear with NULL values	SELECT * FROM Employee E FULL OUTER JOIN Department D ON E.DeptID = D.DeptID;
THETA JOIN	Returns rows that satisfy a specified comparison condition.	Uses comparison operators (=, <, >, <=, >=, !=)	Rows satisfying the given condition	Depends on the comparison condition	SELECT * FROM Employee E JOIN Department D ON E.DeptID > D.DeptID;
NATURAL JOIN	Automatically joins tables based on columns having the same name and compatible data types.	No ON clause required	Matching rows based on common column(s)	Excluded	SELECT * FROM Employee NATURAL JOIN Department;
CROSS JOIN	Produces the Cartesian product of two tables.	No join condition	Every row from the first table is combined with every row from the second table	Not applicable	SELECT * FROM Employee CROSS JOIN Department;



Thank You !